# LAB 02

→ Objectives

The objective of this lab is to introduce you to C programming by developing the following programs and debugging them:

I) Go over the solution of Lab01 and discuss differences and similarities between C and Java.
II) Introduction of Arrays in C.
III) Introduction of GDB – GNU Debugger.
IV) Write a simple insertion sort program in C, compile and execute the program.

→ Exercise

*Task 1:*

☒ First, we will go over the solution of Lab01 and compare the C and Java code:

Similarities and differences between programming in Java and C using this code

Differences:

| **C Program** | **Java Program** |
|---|---|
| no need to define class | need to define class |
| no restriction | file_name should be same as class name, prefer to be CamelCase letter |
| file extensions are "*.c", "*.h" | file extension is "*.java" |
| library access by using keyword "include" | library access by using keywork "import" |
| no need to specify main method as static | need to specify main method as static |
| main method can return int or void | main method can return only void |
| need to define garbage collection | automatic garbage collection |
| types of compilers: GCC C compiler, Turbo C Compiler, Intel C Compiler, Tiny C compiler | types of compilers: Java Programming Language Compiler (javac), Eclipse Compiler for Java (ECJ), GNU Compiler for Java (GCJ) |
| need to compile all dependent files together with main file | any dependent file will automatically compile and re-compile if needed. |

The above is a partial list of differences between our primeNumber C and Java programs, you can expand this list as you learn more about the C programming language.

Similarities:

| **C Program** | **Java Program** |
|---|---|

- both need main method
- syntax for comments /* comment */ & //line comment
- escape sequences like: \n, \t, \b, \r, \\, \'
- logical operators like: &&, ||, !
- syntax for loops
- statement always terminate with ; (semicolon)
- conditional statement is same →

```
if (expression) {
        (statement)
else {
        (statement)
}
```

The above is a partial list of similarities between our primeNumber C and Java programs, you can expand this list as you learn more about the C programming language.

*Task 2:*

Array Syntax:

*type* array_name[array_size];

*type* array_name[] = {element1, element2, …, elementN}

Example:

int array[10];

int array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

☒ write a program to find minimum number in an array:

a) create static array of type double and size 100
b) initialize the array with random number (use the drand48)
c) print the array
d) find the minimum value in the array and print that value

NOTES: Defining the seed point with srand48() in-order to call random generator function drand48(). Now, after every iteration we will assign the new random value to the array.

*Task 3:*

☐ In this task you will be using the GDB –

GNU Debugger to debug your code. GDB is an extremely useful tool when you must debug your program for runtime (e.g., segmentation faults, core dumps, array out-of-bound exceptions) and logical errors.

**Segmentation Fault** → a runtime error that occurs when a program tries to access a memory location that it is not allowed to access, or tries to access it in an impermissible way

**Core Dumps** → a file containing a complete snapshot of a program's working memory, CPU registers, and execution state captured at the exact moment the program terminates abnormally or crashes

To debug C programs using GDB, you must compile your programs with the **-g** option to instruct the compile to generate the executable with source-level debug information. Once your program is compiled with the **-g** option, then you can use GDB to debug your program as shown below:

gdb myexefile

gdb -tui myexefile

The -tui option displays the source code and GDB command prompt in a split screen.

Commonly Used "gdb" Commands:

| **gdb command** | **description** | **example** |
|---|---|---|
| file *executable* | specifies the program to execute (if you did not provide it as an argument to gdb) | file a.out |
| break *[file:]function* | set breakpoint at function (in file) | break main b myfunc |
| break *line* | set breakpoint at line | break 15 b 15 |
| run *[args]* | execute the program with optional command-line arguments | run run 10 20 |
| set *args* | set command-line arguments | set args 10 20 |
| bt | display the program stack (backtrace) | bt |

| | | |
|---|---|---|
| print *expr* | print the value of the expression | print i p i |
| continue | continue program execution | continue c |
| next | execute next program line and step over any function calls in the line | next n |
| step | execute next program line and step into any function calls in the line | step s |
| list | display source lines where it is currently stopped (or lines after the last list command) | list l |
| list *[file]:function* | display source lines from the beginning of the function (in file) | list myfunc<br>l myfunc |
| list *line* | display source lines around the line | list 15<br>l 15 |
| where | display where error occurred | where |
| help *[command]* | display information on using "gdb" or display information on "gdb" command | help<br>help break |
| quit | exit "gdb" | quit |

When you get a segmentation fault, if you compile your program with the -g option and run the program through "gdb" (using: gdb -tui myexefile), you will be able to immediately identify the line that is causing the segmentation fault.

1) gcc -g -o my_program my_program.c
2) gdb ./my_program
3) handle arguments, running, and quitting

→ Lab Assignment #2

☒ Prompt the user to enter the number of array elements (say, N).

☒ Read the number of elements (N).

☒ Implement the insertion sort algorithm.

☒ Print the sorted array.